CSC 337 — Midterm 1 Study Guide

Banner — scope PROVISIONAL. Midterm 1 scope is **not published**. Practice covers the lectures so far, following the syllabus weeks 1–7: Introduction, HTML & CSS, HTML forms, Get & Post, JavaScript, DOM, and the client/server model. The **date is confirmed**: Tue Oct 6, 2026, 75 minutes, in the class period. Do **not** invent Node, Express, MongoDB, or D3 detail; those are weeks 8+.

Canonical maps: `csc337-topic-map.md` · `announced-assessments-inventory-2026-09-24.md`

1. Header

Field	Value	Status
Course	CSC 337 Web Programming (FA26), TuTh 3:30–4:45 PM, The Commons 305	Confirmed
Assessment	Midterm 1	Confirmed
Date	**Tue Oct 6, 2026**	**Confirmed** (D2L syllabus / tracker)
Length	75 minutes, class period	Confirmed (inventory)
Delivery / materials	Closed-book and the delivery method are **not** in the inventory or topic map	**Verify** in class or on D2L
Coverage	Syllabus weeks 1–7 (client side, plus the client/server model)	**PROVISIONAL**
Timing caveat	The DOM (week 6) and client/server (week 7) lectures have **not happened yet**. Client/server is scheduled for the **midterm week itself**, so it may or may not be lectured before Oct 6.	Watch lecture
Guide updated	2026-09-24 (America/Phoenix) — Opus review pass	

All times are **Arizona time (MST)**.

Sources

Source	Role
`csc337-topic-map.md` + inventory	Week topics, date, provisional label
`Web Programming.pdf` (on box)	Client-side overview
`Csc337-fall26-Syllabus.pdf` (on box)	Midterm date and length
Gradescope format notes `csc337-2026-09-24.md`	ICA and assignment formats (not a midterm format)
Assignment 4 topics (D2L)	**Topic overlap only.** No A4 solutions appear in this guide; A4 is open and individual.

2. Topic checklist (learning order)

Order	Section ID	Topic (syllabus week)	Blind ✓	Check ✓	Retake ✓
1	`337-M1-overview`	Web overview (wk 1)			
2	`337-M1-http`	Browsers, URLs, HTTP basics (wk 1)			
3	`337-M1-html`	HTML structure and elements (wk 2)			
4	`337-M1-css`	CSS selectors, cascade, specificity, box model (wk 2)			
5	`337-M1-layout`	CSS layout (wk 2)			
6	`337-M1-forms`	HTML forms (wk 3)			
7	`337-M1-getpost`	**Get & Post** (wk 4) — *new in this review*			
8	`337-M1-js`	JavaScript core and events (wk 5)			
9	`337-M1-dom`	DOM (wk 6 — not yet lectured)			
10	`337-M1-client-server`	Client/server model (wk 7) — conceptual only			
11	`337-M1-integrate`	Integrated HTML/CSS/JS			

3. Plain explanations + worked examples

Overview + HTTP (`337-M1-overview`, `337-M1-http`)

- **Three client-side layers:** HTML is structure and content, CSS is presentation, and JavaScript is behavior.
- **URL parts:** in `https://example.com:443/shop/items?id=7#reviews`, the scheme is `https`, the host is `example.com`, the port is `443`, the path is `/shop/items`, the query is `id=7`, and the fragment is `reviews`. The browser keeps the fragment; it is never sent to the server.
- **HTTP exchange:** the client sends a request (method, path, headers, optional body). The server replies with a response (status code, headers, optional body). HTTPS is HTTP over TLS encryption.
- **Status code families:** 2xx success (200 OK), 3xx redirect (301, 302), 4xx client error (404 Not Found, 403 Forbidden), and 5xx server error (500).

HTML (`337-M1-html`)

- A document has `<!DOCTYPE html>`, then `<html lang="en">` containing `<head>` (`<meta charset>`, `<title>`, CSS `<link>`) and `<body>`.
- Semantic elements (`header`, `nav`, `main`, `section`, `article`, `footer`) describe meaning. `div` and `span` are generic containers; `div` is block-level and `span` is inline.
- Attributes: `id` must be unique on the page, while a `class` can be reused. `<img>` needs `alt` text. `<a href>` makes links.

CSS (`337-M1-css`, `337-M1-layout`)

Specificity is compared as a tuple (ids, classes/attributes/pseudo-classes, elements), from left to right:

Selector	Tuple
`p`	(0,0,1)
`.note`	(0,1,0)
`div.note`	(0,1,1)
`#main`	(1,0,0)
`ul li.active a`	(0,1,3)
`nav > a:hover`	(0,1,2)

Inline `style=""` beats all of these, and `!important` overrides normal rules; avoid relying on it. When specificity ties, the rule that appears **later** wins.

Box model: content, then padding, then border, then margin.

- With `box-sizing: content-box` (the default), `width` sets the content only, so total width = width + left/right padding + left/right border.
- With `border-box`, `width` already includes padding and border.

Layout: normal flow (block elements stack; inline elements flow in a line), then flexbox (`display:flex`, `justify-content`, `align-items`), grid, and positioning (`relative`, `absolute`, `fixed`), as taught.

Forms (`337-M1-forms`)

- `<form action="/search" method="get">` holds the controls. Each control's `name` attribute becomes the parameter key.
- Pair each control with a label: `<label for="email">` matches `<input id="email">`.
- Common input types: `text`, `email`, `number`, `password`, `checkbox`, `radio` (same `name` makes a group), `submit`, plus `<select>` and `<textarea>`.
- Built-in validation attributes include `required`, `min`, `max`, and `pattern`. JavaScript validation can check anything and show a message in the page.

Get & Post (`337-M1-getpost`) — added in this review (syllabus week 4)

	GET	POST
Where the data goes	In the URL query string (`?q=cats&page=2`)	In the request body
Visible and bookmarkable?	Yes, and it is cached and kept in history	No
Typical use	Retrieving, search, filters	Creating or changing data, logins, larger payloads
Size	Limited by URL length	Much larger

POST is **not** encryption. Only HTTPS protects the data in transit.

JavaScript (`337-M1-js`)

- **Declarations:** `let` and `const` are block-scoped; `var` is function-scoped. `const` stops reassignment but not mutation of an object's contents.
- **Equality:** `===` is strict (no type conversion). `==` converts types first, so `0 == ""` is `true`. Use `===`.
- **Types:** `+` with any string concatenates (`1 + "2"` gives `"12"`). Other arithmetic converts to numbers (`"3" * 2` gives `6`).
- **Sorting:** `arr.sort()` compares values **as strings** by default, so `[10, 9, 1].sort()` gives `[1, 10, 9]`. For numbers use `arr.sort((a, b) => a - b)`.
- **Parsing input:** `Number("")` is `0`, not `NaN`. Check for empty input separately. `Number("12px")` is `NaN`, while `parseInt("12px")` is `12`.
- **Events:** `el.addEventListener("click", handler)` passes the function itself. Writing `handler()` there calls it immediately, which is a bug.

DOM (`337-M1-dom`) — week 6, not yet lectured

- Select elements with `document.getElementById("x")` or `document.querySelector(".card")`.
- Change content with `el.textContent = "..."`, which is safe plain text. `el.innerHTML = "..."` parses HTML, so avoid it with user input.
- Build elements with `document.createElement("li")`, then `parent.appendChild(li)`.
- Change styles and classes with `el.classList.add("hidden")`, or directly with `el.style.color = "red"`.

Client/server (`337-M1-client-server`) — conceptual only

- The client (browser) sends requests; the server holds data and logic and returns responses.
- A **static** page is the same file for everyone. A **dynamic** response is generated per request.
- Client-side JavaScript runs in the browser, so users can see and change it. Validation there improves the user experience but **must be repeated on the server** for security.
- **Stop here.** Server frameworks (Node, Express) and databases are weeks 8+.

---

4. Practice questions (core)

P1 (recall). Name the three client-side layers and give one responsibility of each.

P2 (explain). What does an HTTP response contain besides the body?

P3 (code). Write a minimal valid HTML page with an external stylesheet `site.css` and a script `main.js` that runs after the page's elements exist.

P4 (CSS). Which is more specific, `div.note` or `#main`? Give both tuples.

P5 (box). An element has `width: 100px`, `padding: 10px`, and `border: 1px solid`, with content-box sizing. What is its total rendered width? What is it under border-box?

P6 (forms). Write a form with a labelled email input (required) and a submit button that sends a GET request to `/subscribe`.

P7 (Get & Post). A search form uses GET with fields `q=red shoes` and `size=9`. Write the URL the browser requests from `/search`. What changes if the form uses POST instead?

P8 (JS). A user types `"12, 3, 100"`. Outline the steps to show the numbers sorted ascending, and name the trap in the default `sort()`.

P9 (explain). Why must input validated in client-side JavaScript be checked again on the server?

P10 (recall). When is Midterm 1, and how long is it?

P11 (explain). Why does this guide avoid inventing Node or MongoDB content for the "client/server" topic?

4b. Gradescope-format and midterm-style practice — Practice recommendation

Where this format comes from. The Gradescope format notes show short ICA checks (10 points each) and multi-page assignments graded per page (Assignment 3: four rubric questions × 25 points, shown as Page 1–4). They do **not** show a midterm format, which is unpublished. The mix below (recall, trace, write code, short explain) is a generic 75-minute written-exam shape. Treat it as a recommendation, not a claimed format.

Timed mock (Sun Oct 4): do R1–R4, T1–T4, and W1–W2 in **45 minutes**, on paper, closed-book.

Short recall (ICA grain)

GS-337-R1. Order by specificity, lowest to highest: `.card`, `div`, `#app`, `div.card`, `#app .card`.

GS-337-R2. What does `el.textContent = "<b>Hi</b>"` display, compared with `el.innerHTML = "<b>Hi</b>"`? Which is safer with user input, and why?

GS-337-R3. Match each status code to its meaning: 200, 301, 404, 500.

GS-337-R4. Give two differences between GET and POST.

Output tracing

GS-337-T1. What does this log?

```
console.log(1 + "2", "3" * 2, 0 == "", 0 === "");
```

GS-337-T2. What does this log?

```
let s = 0;
for (let i = 1; i <= 5; i++) {
  if (i % 2 === 0) continue;
  s += i;
}
console.log(s);
```

GS-337-T3. What does this log, and how do you get numeric order?

```
const a = [10, 9, 1, 100];
a.sort();
console.log(a);
```

GS-337-T4. After one full outer pass (`i = 0`) of this bubble sort, what is `a`?

```
const a = [5, 2, 4, 1];
for (let i = 0; i < a.length; i++)
  for (let j = 0; j < a.length - 1 - i; j++)
    if (a[j] > a[j + 1]) { const t = a[j]; a[j] = a[j + 1]; a[j + 1] = t; }
```

GS-337-T5. Count letters case-insensitively in `"Mississippi"`. What are the counts?

Write code (DOM + events)

GS-337-W1. Given `<input id="n"> <button id="go">Double</button> <p id="out"></p>`, write JavaScript so that clicking the button shows twice the number, or `Not a number` for empty or invalid input.

GS-337-W2. Write `validZip(s)`, which returns `true` only for exactly 5 digits. This is a practice spec, deliberately different from any assignment.

Short explain

GS-337-E1. A number-guessing page picks a secret from 1 to 100. With the best strategy (always guess the middle), what is the maximum number of guesses needed? Why?

GS-337-E2. You submit a multi-file web assignment on Gradescope. What two checks prevent a zero for "missing file"? This is process, not content.

ICA habit (process)

The format notes show every submitted ICA earned full credit, but two ICAs had **no submission**, an upload miss rather than a content gap. The habit to build: upload each ICA before 11:00 PM on lecture day. No extra ICA content drilling is needed.

5. ANSWER KEY

Core keys

A1. HTML gives structure and content, CSS gives presentation and layout, and JavaScript gives behavior and interactivity.

A2. A status line (code and reason, such as `200 OK`) and headers (such as `Content-Type`, `Set-Cookie`, and caching). The body is the payload.

A3.

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="utf-8">
  <title>Demo</title>
  <link rel="stylesheet" href="site.css">
  <script src="main.js" defer></script>
</head>
<body>
  <h1>Hi</h1>
</body>
</html>
```

`defer` in `<head>` works, and so does placing the `<script>` as the last element inside `<body>`.

A4. `#main` is (1,0,0) and `div.note` is (0,1,1). Ids beat any number of classes, so `#main` is more specific.

A5. Content-box: 100 + 2·10 + 2·1 = **122px**. Border-box: **100px**, with content shrunk to 100 − 20 − 2 = 78px.

A6.

```
<form action="/subscribe" method="get">
  <label for="em">Email</label>
  <input type="email" id="em" name="email" required>
  <button type="submit">Subscribe</button>
</form>
```

A7. `/search?q=red+shoes&size=9`. The space is encoded as `+` or `%20`. With POST, the URL is just `/search`, and `q=red+shoes&size=9` travels in the request body, so it is not visible in the URL or history.

A8. Steps:

1. Split on `,`.
2. Trim each piece and convert it with `Number`.
3. Reject `NaN` values.
4. Sort with `(a, b) => a - b`.
5. Display the result with `textContent` or a textarea value.

The trap: the default `sort()` compares strings, so it gives `[100, 12, 3]`.

A9. Client code runs on the user's machine and can be bypassed, for example by disabling JavaScript or sending a request directly. Only server checks are trustworthy. Client checks exist for fast feedback.

A10. Tue Oct 6, 2026, in the 75-minute class period (3:30–4:45 PM Arizona).

A11. The scope is provisional, and the only published topic label is "client/server model" (week 7). Node, Express, and MongoDB are scheduled for weeks 8+ and have not been taught, so inventing detail would mean studying the wrong material.

Gradescope-format keys

GS-337-R1. `div` (0,0,1) < `.card` (0,1,0) < `div.card` (0,1,1) < `#app` (1,0,0) < `#app .card` (1,1,0).

GS-337-R2. `textContent` shows the literal text `<b>Hi</b>`. `innerHTML` renders a bold **Hi**. `textContent` is safer, because `innerHTML` with user input can inject markup or scripts (XSS).

GS-337-R3. 200 = OK. 301 = moved permanently (redirect). 404 = not found. 500 = internal server error.

GS-337-R4. Any two of:

- GET puts data in the URL; POST puts it in the body.
- GET is visible, bookmarkable, and cached; POST is not.
- GET is for retrieval; POST is for creating or changing data.
- GET is limited by URL length.

GS-337-T1. `12 6 true false`

GS-337-T2. `9` (1 + 3 + 5).

GS-337-T3. `[1, 10, 100, 9]`, because the values are compared as strings. Fix: `a.sort((x, y) => x - y)` gives `[1, 9, 10, 100]`.

GS-337-T4. `[5,2,4,1]` → `[2,5,4,1]` → `[2,4,5,1]` → `[2,4,1,5]`. After the pass, `a` is **`[2, 4, 1, 5]`**, and the largest value has bubbled to the end.

GS-337-T5. m: 1, i: 4, s: 4, p: 2 (11 letters total).

GS-337-W1.

```
document.getElementById("go").addEventListener("click", () => {
  const raw = document.getElementById("n").value.trim();
  const out = document.getElementById("out");
  const v = Number(raw);
  out.textContent = (raw === "" || Number.isNaN(v)) ? "Not a number" : String(v * 2);
});
```

The key points are passing the handler without `()`, trimming the input, catching the empty-string trap, and using `textContent`.

GS-337-W2.

```
function validZip(s) {
  return /^\d{5}$/.test(s);
}
```

A loop that checks `s.length === 5` and that every character is between `'0'` and `'9'` is also fine.

GS-337-E1. 7 guesses. Each guess halves the remaining range, and $2^6 = 64 < 100 \leq 128 = 2^7$.

GS-337-E2. (1) Upload **every** required file, with filenames exactly as the assignment PDF lists them. (2) Open the submission page afterwards to confirm all files are there. Also check the deadline: assignments are due at **3:30 PM**, not 11:59 PM.

---

6. Common mistakes + readiness

- Invalid nesting, or a `label` whose `for` doesn't match the input's `id`.
- Fighting specificity with `!important` instead of writing a clearer selector.
- Assuming border-box sizing when the default is content-box.
- Thinking POST is "secure." Only HTTPS encrypts.
- Using `==` instead of `===`, or forgetting that `Number("")` is `0`.
- Sorting numbers with the default `sort()`.
- `addEventListener("click", f())`, which calls `f` immediately.
- Using `innerHTML` with user text.

Readiness: a blind pass over all **11** section IDs, the 45-minute paper mock (§4b), and a light review on Monday night — no cram.

---

7. Day-by-day prep (Tue/Thu light; no day-before cram)

Day	Focus	IDs	Notes
Tue Sep 29	—	—	A4 due 3:30 PM. This guide does not cover A4.
Wed Sep 30	HTTP, URLs, Get & Post	http, getpost	Short
Fri Oct 2	HTML and CSS review	html, css, layout	Deep day
Sat Oct 3	Forms, JS, and DOM (after the week-6 lecture)	forms, js, dom	Deep day
Sun Oct 4	**45-minute paper mock** (§4b), then review	integrate	Blind, then check
Mon Oct 5	Even rotation over all IDs; client/server concepts	all `337-M1-*`	Deep day, but stop by evening
Tue Oct 6	Calm buffer only	—	Midterm in class at 3:30 PM

---

8. Changes in this review (2026-09-24)

- Added **Get & Post** (syllabus week 4), which was missing from the checklist. It now has an explanation, practice items, and keys.
- **Removed Assignment 4–specific solution material:** the working email validator for A4's exact rule, A4's exact validation list, and A4's filenames. A4 is open and individual-only. The drills are re-specified: ZIP validator, letter counts, a DOM "double" button, and guess-count reasoning.
- Added a timing caveat: DOM and client/server have not been lectured yet, and client/server falls in the midterm week itself.
- Marked "closed-book" as **verify**, because the inventory and topic map don't state it.
- Rebuilt the specificity explanation with tuples, and added JavaScript traps (`==`, `Number("")`, default `sort`, event handler), DOM safety, and status codes.
- Fixed the readiness line: it said "all eight IDs" when there were ten; there are now eleven.
- Changed times to Arizona time.

9. Disclaimer

The Gradescope format notes describe ICA and assignment formats only. The midterm format and blueprint are **not published**, so the §4b shape is a recommendation.

Study aid only — not for submitted work (assignments or exams). Follow CSC 337 and UA AI and academic-integrity policies.